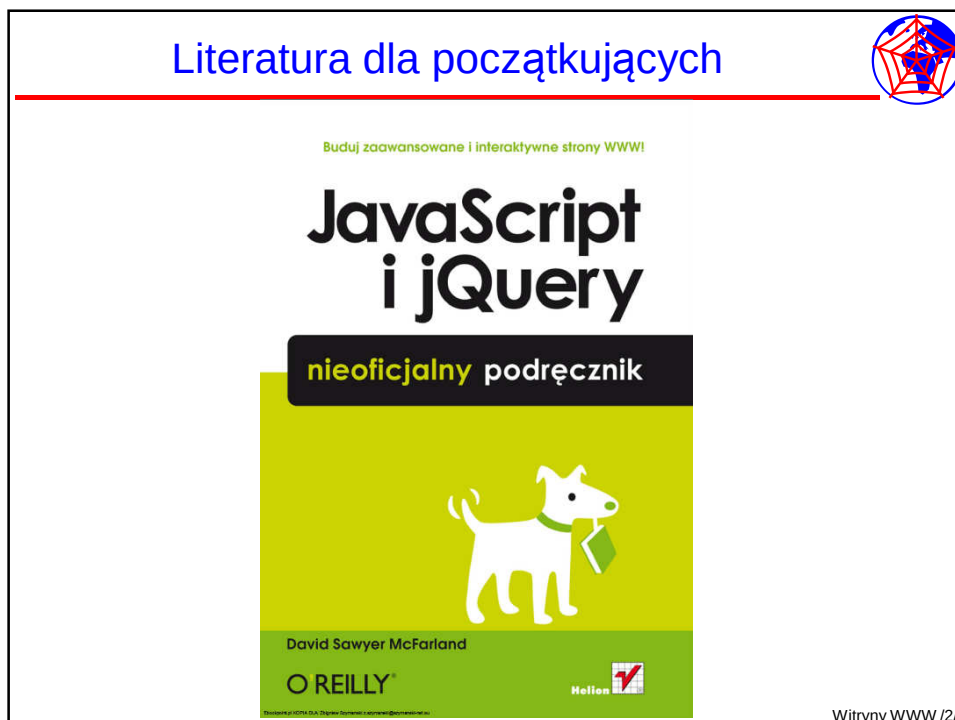




Javascript - co to jest?



- ❖ Umożliwia dodanie elementów interaktywnych do strony WWW
- ❖ Javascript jest językiem skryptowym (wykonywanym po stronie przeglądarki internetowej)
- ❖ Javascript jest językiem interpretowanym
- ❖ Skrypty Javascript są zazwyczaj wbudowane w kod HTML

Witryny WWW /3/

Co potrafi Javascript?



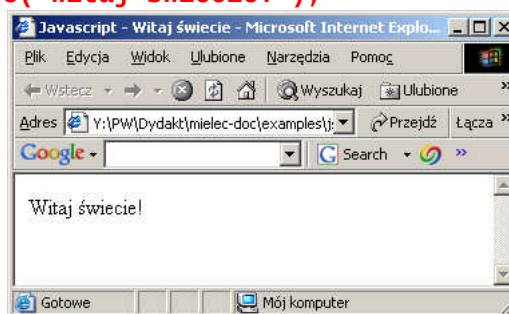
- ❖ Javascript umożliwia wstawienie dynamicznej treści do strony WWW.
`document.write("<h1>" + name + "</h1>")`
- ❖ Javascript umożliwia reakcję na zdarzenia - np. zakończenie pobierania strony, kliknięcie na elemencie HTML
- ❖ Javascript może czytać i zapisywać elementy HTML - umożliwia pobieranie i modyfikację zawartości strony WWW
- ❖ Javascript umożliwia sprawdzenie poprawności danych przed wysłaniem ich na serwer
- ❖ Javascript umożliwia identyfikację przeglądarki WWW

Witryny WWW /4/

Javascript - Witaj Świecie



```
<html>
<head>
  <title>Javascript - Witaj świecie</title>
</head>
<body>
  <script type="text/javascript">
    document.write("Witaj świecie!");
  </script>
</body>
</html>
```



Witryny WWW /5/

Gdzie umieścić skrypt - sekcja head



❖ W sekcji head

```
<html>
<head>
  <script type="text/javascript">
    ...
  </script>
</head>
<body>

</body>
</html>
```

❖ Skrypty wykonywane w momencie wywołania lub zajścia określonego zdarzenia. Umieszczenie skryptu w sekcji head gwarantuje, że zostanie załadowany przed wykonaniem.

Witryny WWW /6/

Gdzie umieścić skrypt - sekcja *body*



- ❖ W sekcji *body*

```
<html>
  <head>

  </head>
  <body>
    <script type="text/javascript">
      ...
    </script>
  </body>
</html>
```

- ❖ Skrypty wykonywane w momencie ładowania strony. Skrypty tworzą zawartość strony WWW.

Witryny WWW /7/

Gdzie umieścić skrypt - zewnętrzny plik



- ❖ W zewnętrznym pliku

```
<html>
  <head>
    <script src="xxx.js"></script>
  </head>
  <body>
    <script src="xxx.js"></script>
  </body>
</html>
```

- ❖ Gdy skrypt jest wykorzystywany na kilku stronach najlepiej jest umieścić go w zewnętrznym pliku o rozszerzeniu *.js*.
- ❖ W pliku ze skryptem nie umieszcza się znacznika *<script>*

Witryny WWW /8/

Javascript - Zmienne



- ❖ Zmienne służą do przechowywania informacji.
- ❖ Zawartość zmiennej może się zmienić w trakcie wykonywania skryptu.
- ❖ Do zmiennej odwołujemy się wykorzystując nazwę zmiennej.
- ❖ Zasady tworzenia nazw zmiennych:
 - ⌘ rozróżniane są wielkie i małe litery - np. **wynik** i **Wynik** to dwie różne zmienne
 - ⌘ nazwa zmiennej musi się rozpoczynać od litery lub znaku podkreślenia

Witryny WWW /9/

Deklaracja zmiennej, przypisanie wartości



- ❖ Deklaracja zmiennej
 - ⌘ z użyciem słowa kluczowego **var**
var wynik = wartosc
 - ⌘ bez użycia **var**
wynik = wartosc
- ❖ Przypisanie wartości
 - var nazwisko = "kowalski"**
 - ⌘ bez użycia **var**
nazwisko = "kowalski"

Witryny WWW /10/

Czas życia zmiennych



- ❖ Zmienne zadeklarowane wewnątrz funkcji
 - ⌘ są dostępne tylko w obrębie danej funkcji
 - ⌘ po wyjściu z funkcji są niszczone
 - ⌘ może istnieć wiele zmiennych lokalnych o tej samej nazwie - każda w innej funkcji
- ❖ Zmienne zadeklarowane poza funkcją
 - ⌘ mogą być wykorzystane we wszystkich funkcjach na danej stronie
 - ⌘ czas życia rozpoczyna się w momencie deklaracji i kończy wraz z zamknięciem strony

Witryny WWW /11/

Instrukcje warunkowe



- ❖ **if ...** - (jeżeli ...) instrukcja wykorzystywana aby wykonać kod, gdy warunek jest prawdziwy
- ❖ **if ... else ...** - (jeżeli ... w przeciwnym przypadku ...) instrukcja wykorzystywana aby wykonać kod, gdy warunek jest prawdziwy oraz by wykonać inny fragment kodu gdy warunek jest fałszywy
- ❖ **if ... else if ... else ...** - (jeżeli ... w przeciwnym przypadku ... jeżeli ...) instrukcja wykorzystywana aby wykonać jeden z kilku fragmentów kodu
- ❖ **switch** - (wybierz, przełącz) instrukcja wykorzystywana aby wykonać jeden z kilku fragmentów kodu

Witryny WWW /12/

Składnia instrukcji if ... else ...



```
if (warunek1)
{
    kod do wykonania jeśli warunek1 jest prawdziwy
}
else if (warunek2)
{
    kod do wykonania jeśli warunek2 jest prawdziwy
}
else
{
    kod do wykonania jeśli żaden z poprzednich warunków nie
    był prawdziwy
}
```

Witryny WWW /13/

If - przykład



```
<script type="text/javascript">
var d=new Date();
var time=d.getHours();
if (time<10)
{
    document.write("<b>Miłego poranka</b>");
}
</script>
```

```
<script type="text/javascript">
var d=new Date();
var time=d.getHours();

if (time==11)
{
    document.write("<b>Czas na lunch!</b>");
}
</script>
```

Witryny WWW /14/

If ... else ... - przykład



```
<script type="text/javascript">

var d = new Date();
var time = d.getHours();

if (time < 10)
{
    document.write("Miłego poranka!");
}
else
{
    document.write("Dzień dobry!");
}
</script>
```

Witryny WWW /15/

If ... else if ... else ... - przykład



```
<script type="text/javascript">
var d = new Date();
var time = d.getHours();
if (time<10)
{
    document.write("<b>Miłego poranka</b>");
}
else if (time>10 && time<16)
{
    document.write("<b>Dzień dobry!</b>");
}
else
{
    document.write("<b>Dobry wieczór</b>");
}
</script>
```

Witryny WWW /16/

Składnia instrukcji switch



```
switch(n)
{
  case 1:
    wykonaj blok 1
    break
  case 2:
    wykonaj blok 2
    break
  ...
  default:
    kod wykonywany jeśli n nie spełnia
    żadnego z powyższych warunków
}
```

Witryny WWW /17/

switch - przykład



```
<script type="text/javascript">
//Sunday=0, Monday=1, Tuesday=2, itd.
var d=new Date();
theDay=d.getDay();
switch (theDay)
{
  case 5:
    document.write("Nareszcie piątek");
    break;
  case 6:
    document.write("Sobota :)");
    break;
  case 0:
    document.write("Niedziela");
    break;
  default:
    document.write("Czekam na weekend!");
}
</script>
```

Witryny WWW /18/

Operatory arytmetyczne			
Operator	Opis	Przykład	Wynik
+	dodawanie	x=5 y=2 x+y	7
-	odejmowanie	x-y	3
*	mnożenie	x*y	10
/	dzielenie	5/2	2.5
%	reszta z dzielenia	5%2	1
		10%8	2
		10%2	0
++	inkrementacja	x++	x==6
--	dekrementacja	x--	x==4

Witryny WWW /19/

Operatory przypisania		
Operator	Przykład	Równoważne operacji ...
=	x=y	
+=	x+=y	x=x+y
-=	x-=y	x=x-y
=	x=y	x=x*y
/=	x/=y	x=x/y
%=	x%=y	x=x%y

Witryny WWW /20/

Operatory porównania



Operator	Opis	Przykład
<code>==</code>	jest równy	<code>5==8</code> wyrażenie fałszywe
<code>===</code>	jest równy (sprawdza wartość i typ)	<code>x=5</code> <code>y= "5"</code> <code>x==y</code> wyrażenie prawdziwe <code>x===y</code> wyrażenie fałszywe
<code>!=</code>	nie równa się	<code>5!=8</code> wyrażenie prawdziwe
<code>></code>	większe niż	<code>5>8</code> wyrażenie fałszywe
<code><</code>	mniejsze niż	<code>5<8</code> wyrażenie prawdziwe
<code>>=</code>	jest mniejsze lub równe	<code>5>=8</code> wyrażenie fałszywe
<code><=</code>	jest mniejsze lub równe	<code>5<=8</code> wyrażenie prawdziwe

Witryny WWW /21/

Operatory logiczne



Operator	Opis	Przykład
<code>&&</code>	i	<code>x=6; y=3;</code> <code>(x<10 && y>1)</code> wyrażenie prawdziwe
<code> </code>	lub	<code>(x==5 && y==5)</code> wyrażenie fałszywe
<code>!</code>	nie	<code>!(x==y)</code> wyrażenie prawdziwe

Witryny WWW /22/

Operatory na łańcuchach znakowych



```
txt1="Bardzo ładny"  
txt2="dzień!"  
txt3=txt1+txt2
```

❖ Wynik: txt3 == "Bardzo ładnydzień!"

```
txt1="Bardzo ładny"  
txt2="dzień!"  
txt3=txt1+" "+txt2
```

```
txt1="Bardzo ładny"  
txt2="dzień!"  
txt3=txt1+txt2
```

❖ Wynik: txt3 == "Bardzo ładny dzień!"

Witryny WWW /23/

Operator warunkowy



❖ Składnia:

```
zmienna = (warunek) ? wartość1 : wartość2
```

❖ Przykład:

```
pozdrawienie =  
(gosc=="PREZ")?"Szanowny Prezydencie":"Szanowny Gościu"
```

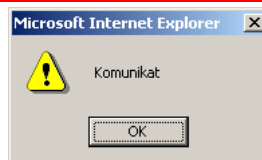
Witryny WWW /24/

Popup box



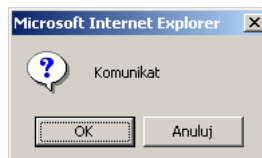
❖ Alert box

```
alert("Komunikat");
```



❖ Confirm box - zwraca: true / false

```
confirm("Komunikat");
```



❖ Prompt box - zwraca: wartość / null

```
prompt("Komunikat", "Wartość domyślna");
```



Witryny WWW /25/

Funkcje



❖ Funkcja jest blokiem kodu, który można wielokrotnie wykonać wywołując funkcję w skrypcie lub poprzez zdarzenie.

❖ Funkcje umieszczamy w sekcji *head*.

❖ Składnia:

```
function nazwafunkcji(par1, par2, ..., parX)
{
    kod funkcji
    return wynik;
}
```

Witryny WWW /26/

Zmienne w funkcjach



Zmienna lokalna w funkcji

```
var message = 'Poza funkcją';

function warning() {
  var message = 'W funkcji';
  alert(message); // 'Poza funkcją'
}
warning();
alert(message); // 'W funkcji'
```

Rysunek 3.9. Przy przypisywaniu wartości do zmiennych w funkcji należy pamiętać o pewnym drobnym, ale istotnym szczególe. Jeśli chcesz, aby zmienna była dostępna tylko w funkcji, pamiętaj o dodaniu słowa kluczowego `var` przy tworzeniu tej zmiennej (górny kod). Jeśli go nie użyjesz, zapiszesz nową wartość w zmiennej globalnej (kod dolny)

Zmienna globalna w funkcji

```
var message = 'Poza funkcją';

function warning() {
  message = 'W funkcji';
  alert(message); // 'W funkcji'
}
warning();
alert(message); // 'W funkcji'
```

Źródło: David Sawyer McFarland „JavaScript i jQuery – nieoficjalny podręcznik”, Helion 2012

Witryny WWW /27/

Funkcje - przykład /1/



```
<html>
<head>
  <script type="text/javascript">
    function displaymessage()
    {
      alert("Hello World!");
    }
  </script>
</head>
<body>
  <form>
    <input type="button" value="Click me!"
      onclick="displaymessage()" >
  </form>
</body>
</html>
```

Witryny WWW /28/

Funkcje - przykład /2/



❖ Definicja funkcji:

```
<script type="text/javascript">
  function iloczyn(a,b)
  {
    x=a*b;
    return x;
  }
</script>
```

❖ Wywołanie funkcji:

```
wynik=iloczyn(2,3);
```

Witryny WWW /29/

Tablice – dodawanie elementów



Metoda	Wyjściowa tablica	Przykładowy kod	Wynikowa tablica	Opis
Właściwość length	var p = [0,1,2,3]	p[p.length]=4	[0,1,2,3,4]	Dodaje jedną wartość na koniec tablicy.
push()	var p = [0,1,2,3]	p.push(4,5,6)	[0,1,2,3,4,5,6]	Dodaje jeden lub więcej elementów na koniec tablicy.
unshift()	var p = [0,1,2,3]	p.unshift(4,5)	[4,5,0,1,2,3]	Wstawia jeden lub więcej elementów na początek tablicy.

Źródło: David Sawyer McFarland „JavaScript i jQuery – nieoficjalny podręcznik”, Helion 2012

Witryny WWW /30/

Tablice – usuwanie elementów



Metoda	Wyjściowa tablica	Przykładowy kod	Wynikowa tablica	Opis
<code>pop()</code>	<code>var p = [0,1,2,3]</code>	<code>p.pop()</code>	<code>[0,1,2]</code>	Usuwa z tablicy ostatni element.
<code>shift()</code>	<code>var p = [0,1,2,3]</code>	<code>p.shift()</code>	<code>[1,2,3]</code>	Usuwa z tablicy pierwszy element.

Źródło: David Sawyer McFarland „JavaScript i jQuery – nieoficjalny podręcznik”, Helion 2012

Witryny WWW /31/

Tablice – przykład



```
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
<title>Tworzenie i używanie tablic</title>
<script>
var authors = [ 'Ernest Hemingway', 'Charlotte Bronte', 'Dante Alighieri',
                'Emily Dickinson'];
</script>
</head>
<body>
<h1>Tworzenie i używanie tablic</h1>
<script>
    document.write('<p>Pierwszy autor to <strong>');
    document.write(authors[0] + '</strong></p>');
    document.write('<p>Ostatni autor to <strong>');
    document.write(authors[authors.length-1] + '</strong></p>');
    authors.unshift('Stan Lee');
    document.write('<p>Prawie zapomniałem o autorze <strong>');
    document.write(authors[0]);
    document.write('</strong></p>');
</script>
</body>
</html>
```

Witryny WWW /32/

Pętla - instrukcja for



❖ Pętle są wykorzystywane do wielokrotnego wykonania pewnego fragmentu kodu w zależności od określonego warunku.

❖ Składnia pętli for:

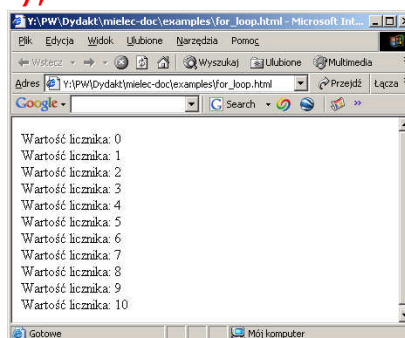
```
for (licznik=start; licznik<=koniec; licznik=licznik+skok)
{
    kod do wykonania
}
```

Witryny WWW /33/

Pętla for - przykład



```
<html>
<body>
  <script type="text/javascript">
    var i=0;
    for (i=0; i<=10; i++)
    {
      document.write("Wartość licznika: " + i);
      document.write("<br>");
    }
  </script>
</body>
</html>
```



Witryny WWW /34/

Pętla while, do ... while



❖ Pętla while jest wykorzystywana aby wykonać kod podczas gdy określony warunek jest spełniony.

❖ Składnia pętli while:

```
while (warunek)
{
    kod do wykonania
}
```

❖ Składnia pętli do ... while:

```
do {
    kod do wykonania
} while (warunek)
```

Witryny WWW /35/

Pętla while - przykład



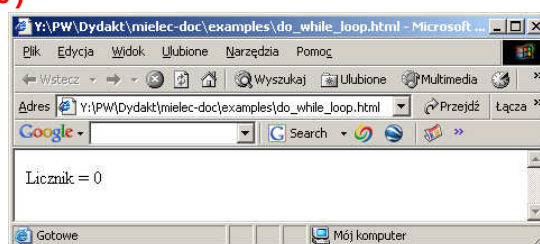
```
<html>
<body>
  <script type="text/javascript">
    var i=0;
    while (i<=10)
    {
      document.write("Licznik = " + i);
      document.write("<br>");
      i=i+1
    }
  </script>
</body>
</html>
```

Witryny WWW /36/

Pętla do ... while - przykład



```
<html>
<body>
  <script type="text/javascript">
    var i=0;
    do
      {
        document.write("Licznik = " + i);
        document.write("<br>");
        i=i+1;
      } while (i<0)
  </script>
</body>
</html>
```



Witryny WWW /37/

Pętla do...while – przykład /2/



```
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
<title>Stosowanie instrukcji warunkowych</title>
<script>
  do {
    var luckyNumber = prompt('Jaka jest Twoja szczęśliwa liczba?', '');
    luckyNumber = parseInt(luckyNumber, 10);
  } while (isNaN(luckyNumber));
</script>
</head>
<body>
  <h1>Proszę, podaj swoją szczęśliwą liczbę</h1>
  <script>
    if (luckyNumber == 7 ) {
      document.write("<p>7 to także e moja szczęśliwa liczba!<p>");
    } else if (luckyNumber == 13 || luckyNumber == 24) {
      document.write("<p>Naprawdę " + luckyNumber + "? To pechowa liczba!<p>");
    } else {
      document.write("<p>Twoja szczęśliwa liczba to " + luckyNumber + "!<p>");
    }
  </script>
</body>
</html>
```

Witryny WWW /38/

Instrukcja break



- ❖ Instrukcja break powoduje przerwanie wykonywania pętli.

```
<html>
<body>
  <script type="text/javascript">
    var i=0;
    for (i=0;i<=10;i++)
    {
      if (i==3){break;}
      document.write("Licznik = " + i);
      document.write("<br>");
    }
  </script>
</body>
</html>
```

Witryny WWW /39/

Instrukcja continue



- ❖ Instrukcja continue przerywa aktualny obieg pętli i kontynuuje pętlę z nową wartością zmiennej sterującej.

```
<html>
<body>
  <script type="text/javascript">
    var i=0
    for (i=0;i<=10;i++)
    {
      if (i==3){continue}
      document.write("Licznik = " + i)
      document.write("<br />")
    }
  </script>
</body>
</html>
```

Witryny WWW /40/

For ... in ...



❖ Instrukcja `for... in...` służy do iterowania po elementach tablicy lub właściwościach obiektu. Kod w nawiasach klamrowych jest wykonywany dokładnie raz dla każdego elementu / właściwości.

❖ Składnia:

```
for (zmienna in obiekt)
{
    kod do wykonania
}
```

Witryny WWW /41/

For... in... - przykład



```
<html>
<body>
  <script type="text/javascript">
    var x;
    var pojazdy = new Array("Saab", "Volvo", "BMW");

    for (x in pojazdy)
    {
      document.write(pojazdy[x] + "<br />");
    }
  </script>
</body>
</html>
```

Witryny WWW /42/

Zdarzenia (Events)



- ❖ Każdy element strony WWW posiada pewien zestaw zdarzeń, które mogą spowodować wykonanie funkcji Javascript.
- ❖ Zdarzenia definiowane są w znacznikach HTML.
- ❖ Zdarzenia zostały wprowadzone w HTML w wersji 4.0
- ❖ Przykłady zdarzeń:
 - ⌘ kliknięcie myszą
 - ⌘ załadowanie strony lub obrazka
 - ⌘ wysłanie formularza
 - ⌘ naciśnięcie klawisza
 - ⌘ wejście do pola wprowadzania tekstu

Witryny WWW /43/

Lista zdarzeń /1/



atrybut	opis
onabort	przerwanie ładowania obrazka
onblur	utrata focus'u przez element
onchange	zmiana zawartości pola
onclick	kliknięcie myszą na obiekcie
ondblclick	podwójne kliknięcie myszą na obiekcie
onerror	błąd podczas ładowania dokumentu lub obrazka
onfocus	otrzymanie focus'u przez element
onkeydown	wciśnięcie klawisza na klawiaturze
onkeypress	wciśnięcie lub przytrzymanie klawisza na klawiaturze
onkeyup	zwolnienie klawisza na klawiaturze

Witryny WWW /44/

Lista zdarzeń /2/



atrybut	opis
onload	zakończenie ładowania strony lub obrazka
onmousedown	wciśnięcie przycisku myszy
onmousemove	przesunięcie myszy
onmouseout	przesunięcie kursora myszy poza element
onmouseover	przesunięcie kursora myszy nad elementem
onmouseup	zwolnienie przycisku myszy
onreset	kliknięcie na przycisk reset
onresize	zmiana rozmiaru okna lub ramki
onselect	zaznaczenie tekstu
onsubmit	kliknięcie na przycisk Submit
onunload	użytkownik wychodzi ze strony

Witryny WWW /45/

Zdarzenia /1/



- ❖ Zdarzenie onload może być wykorzystane do wykrycia typu i wersji przeglądarki internetowej w celu załadowania właściwej wersji strony.
- ❖ Zdarzenia onload, onunload mogą posłużyć do obsługi plików cookies (np. do zapamiętania danych o użytkowniku przy wyjściu ze strony i ich ponownego wykorzystania przy ponownym wejściu na stronę).
- ❖ onfocus, onblur, onchange są często wykorzystywane do sprawdzania poprawności pól formularzy

```
<input type="text" size="30" id="email" onchange="checkEmail()">;
```

Witryny WWW /46/

Zdarzenia /2/



- ❖ onsubmit może posłużyć do sprawdzenia poprawności wszystkich pól formularza.

```
<form method="post" action="xxx.htm"
onsubmit="return checkForm()">
```

- ❖ onmouseover może być wykorzystane do realizacji animowanych przycisków.

```
<a href="http://www.w3schools.com"
onmouseover="alert('Zdarzenie onMouseOver');return
false">

</a>
```

Witryny WWW /47/

Obsługa błędów w skryptach



- ❖ W przypadku wystąpienia błędów w czasie wykonania skryptu może pojawić się komunikat: „Do you wish to debug?”. Komunikat być może przydatny dla autora strony, ale nie dla odbiorcy.

- ❖ Sposoby przechwycenia sytuacji błędnych:

- ⌘ fraza **try... catch** - wymaga nowszych przeglądarek (IE5+, Netscape 6, Mozilla 1.0)

- ⌘ obsługa zdarzenia **onerror** - dostępne od Netscape 3

Witryny WWW /48/

try ... catch



```
try
{
    //Nadzorowany kod
}
catch(err)
{
    //Obsługa błędów
}
```

Witryny WWW /49/

try ... catch - przykład /1/

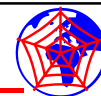


```
<html>
<head>
<script type="text/javascript">
var txt=""
function message()
{
    try
    {
        addalert("Welcome guest!")
    }
    catch(err)
    {
        txt="Wystąpił błąd na stronie.\n\n"
        txt+="Opis błędu: " + err.description + "\n\n"
        txt+="Naciśnij OK aby kontynuować.\n\n"
        alert(txt)
    }
}
</script>
</head>
<body>
<input type="button" value="View message"
    onclick="message()" />
</body></html>
```

Błąd! Zamiast
alert jest
addalert

Witryny WWW /50/

try ... catch - przykład /2/



```
<head>
<script type="text/javascript">
var txt=""
function message()
{
  try
  {
    adddler("Welcome guest!")
  }
  catch(err)
  {
    txt="Wystąpił błąd na tej stronie.\n\n"
    txt+="Kliknij na OK aby kontynuować lub,\n"
    txt+="Anuluj aby powrócić do strony domowej.\n\n"
    if(!confirm(txt))
    {document.location.href="http://www.w3schools.com/"}
  }
}
</script>
</head>
<body>
<input type="button" value="View message"
  onclick="message()" />
</body>
```

Błąd! Zamiast
alert jest
adddler

Witryny WWW /51/

throw



- ❖ Polecenie throw pozwala na wywołanie wyjątku (ang. exception). Użyte razem z frazą try... catch umożliwia sterowanie wykonaniem skryptu oraz generowanie dokładnych komunikatów o zachodzących błędach.
- ❖ Składnia polecenia throw:
throw(wyjątek)
 - ⌘ wyjątek może być: łańcuchem znakowym (string), liczbą całkowitą (integer), wartością Boolean lub obiektem.

Witryny WWW /52/

throw - przykład



```
<html>
<body>
<script type="text/javascript">
var x=prompt("Podaj liczbę od 0 do 10:", "")
try
{
  if(x>10)
    throw "Err1"
  else if(x<0)
    throw "Err2"
}
catch(er)
{
  if(er=="Err1")
    alert("Błąd! Liczba za duża")
  if(er == "Err2")
    alert("Błąd! Liczba za mała")
}
</script>
</body>
</html>
```

Witryny WWW /53/

Obsługa zdarzenia onerror



❖ Zdarzenie onerror pozwala na przechwytywanie błędów występujących w skryptach.

❖ Składnia:

onerror=handleErr

```
function handleErr(msg,url,line)
{
  //Tutaj obsługa błędu
  return true or false
}
```

❖ zwracana wartość:

- ⌘ true - przeglądarka nie wyświetli standardowego komunikatu o błędzie
- ⌘ false - przeglądarka wyświetli standardowy komunikat o błędzie

Witryny WWW /54/

Obsługa zdarzenia onerror - przykład



```
<head>
<script type="text/javascript">
  onerror=handleErr
  var txt=""
  function handleErr(msg,url,l)
  {
    txt="Wystąpił błąd na stronie.\n\n"
    txt+="Błąd: " + msg + "\n"
    txt+="URL: " + url + "\n"
    txt+="Linia: " + l + "\n\n"
    txt+="Kliknij OK aby kontynuować.\n\n"
    alert(txt)
    return true
  }
  function message()
  {
    addalert("Witaj wędrowcze!")
  }
</script>
</head>
<body>
  <input type="button" value="View message"
    onclick="message()" />
</body>
```

Błąd! Zamiast
alert jest
addalert

Witryny WWW /55/

Znaki specjalne w Javascript



Kod Opis

\'	pojedynczy cudzysłów
\"	podwójny cudzysłów
\&	ampersand
\\	backslash
\n	nowa linia
\r	powrót karetki
\t	tab
\b	backspace
\f	form feed

❖ Przykład:

```
var txt="Twoim ulubionym tygodnikiem są \"Plotki\"."
document.write(txt)
```

Witryny WWW /56/

Javascript - uwagi



- ❖ rozróżniane są małe i wielkie litery
- ❖ dodatkowe *znaki białe* są ignorowane

```
name="Jan"
name = "Jan"
```
- ❖ jeden łańcuch znakowy można umieścić w dwóch liniach kodu wykorzystując znak backspace

```
document.write("Witaj \
świecie!")
```
- ❖ komentarze
 - ⌘ pojedyncza linia //
 - ⌘ kilka linii komentarza /* */

Witryny WWW /57/

Obsługa cookies w Javascript /1/



```
function setCookie(c_name,value,expiredays)
{
  var exdate=new Date()
  exdate.setDate(exdate.getDate()+expiredays)
  document.cookie=c_name+ "=" +escape(value)+
    ((expiredays==null) ? "" : ";expires="+exdate)
}
```

Witryny WWW /58/

Obsługa cookies w Javascript /2/



```
function getCookie(c_name)
{
  if (document.cookie.length>0)
  {
    c_start=document.cookie.indexOf(c_name + "=")
    if (c_start!=-1)
    {
      c_start=c_start + c_name.length+1
      c_end=document.cookie.indexOf(";",c_start)
      if (c_end==-1) c_end=document.cookie.length
      return
        unescape(document.cookie.substring(c_start,c_end))
    }
  }
  return null
}
```

Witryny WWW /59/

Obsługa cookies w Javascript /3/



```
function checkCookie()
{
  username=getCookie('username')
  if (username!=null)
    {alert('Welcome again '+username+'!')}
  else
    {
      username=prompt('Please enter your name:', "")
      if (username!=null||username!="")
        {
          setCookie('username',username,365)
        }
    }
}

❖ <body onLoad="checkCookie()">
```

Witryny WWW /60/

Biblioteka jQuery



- ❖ <http://www.jquery.com>
- ❖ Zalety:
 - ⌘ Mały rozmiar (ok. 90 KB)
 - ⌘ Przyjazna dla projektantów stron WWW – bazuje na znajomości CSS
 - ⌘ Używana na wielu stronach
 - ⌘ Bezpłatna
 - ⌘ Pozwala tworzyć wtyczki
- ❖ Zastosowania: dodawanie nowych treści do strony, modyfikacja kodu, przypisywanie stylów

Witryny WWW /61/

Wykonanie skryptu po załadowaniu strony



```
<html>
<head>
  <title>...</title>
  <script src="jquery-1.6.3.min.js"></script>
  <script>
    $(document).ready(function() {
      //tutaj zawartosc funkcji ready
    }); // koniec funkcji ready
  </script>
</head>
<body>
  ...
</body>
</html>
```

Witryny WWW /62/

Inne biblioteki



WIEDZA W PIGUŁCE

Inne biblioteki JavaScript

jQuery nie jest jedyną dostępną biblioteką napisaną w języku JavaScript. Istnieje wiele, wiele innych. Niektóre z nich zostały zaprojektowane w celu wykonywania ściśle określonych zadań, inne natomiast mają ogólne przeznaczenie i służą do rozwiązywania wszelkich możliwych problemów, jakie mogą napotkać programiści JavaScript. Oto kilka najbardziej popularnych bibliotek.

♦ **Yahoo User Interface Library** (<http://developer.yahoo.com/yui/>) jest projektem prowadzonym przez Yahoo i w praktyce używanym na jej witrynie. Programiści firmy bezustannie poprawiają i rozwijają tę bibliotekę, udostępnili także jej bardzo dobrą dokumentację.

♦ **Dojo Toolkit** (<http://dojotoolkit.org/>) to kolejna biblioteka istniejąca już od bardzo dawna. Daje ogromne możliwości i stanowi kolekcję bardzo wielu plików JavaScript mogących posłużyć do rozwiązania niemal każdego problemu.

♦ **Mootools** (<http://mootools.net/>) jest kolejną biblioteką służącą głównie do tworzenia płynnych animacji i innych efektów wizualnych. Dysponuje doskonałą dokumentacją i wspaniałą witryną.

♦ **Prototype** (<http://www.prototypejs.org/>) to jedna z pierwszych bibliotek JavaScript, jakie się pojawiły. Bardzo często jest używana wraz z dodatkową biblioteką *scriptaculous* (<http://script.aculo.us/>), pozwalającą tworzyć animacje i inne efekty wizualne.

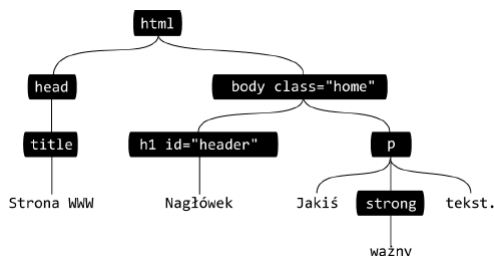
Źródło: David Sawyer McFarland „JavaScript i jQuery – nieoficjalny podręcznik”, Helion 2012

Witryny WWW /63/

Reprezentacja dokumentu HTML - DOM



```
<!DOCTYPE HTML>
<html>
<head>
<meta charset="UTF-8">
<title>Strona WWW</title>
</head>
<body class="home">
<h1 id="header">Nagłówek</h1>
<p>Jakiś <strong>ważny</strong> tekst.</p>
</body>
</html>
```



```
document.getElementById("header");
document.getElementsByTagName("p");
```

Źródło: David Sawyer McFarland „JavaScript i jQuery – nieoficjalny podręcznik”, Helion 2012

Witryny WWW /64/

jQuery – pobieranie elementów /1/



- ❖ przeszukanie DOM według zadanych kryteriów

```
jQuery(selector);  
$(selector);
```

- ❖ Selektory identyfikatorów

```
<p id="message">Komunikat specjalny</p>  
var mesgPara=$("#message");
```

- ❖ Selektory elementów

```
var linksList=$("#a");
```

Witryny WWW /65/

jQuery – pobieranie elementów /2/



- ❖ Selektory klas

```
$(".submenu");
```

- ❖ Selektory elementów potomnych

```
<ul id="navBar">
```

```
$("#navBar a")
```

wybiera wszystkie znaczniki a
znajdujące się wewnątrz listy

- ❖ Selektory dzieci

```
$("body > p")
```

wybiera wszystkie znaczniki p
bezpośrednio w elemencie body

Witryny WWW /66/

jQuery – pobieranie elementów /3/



❖ Selektory elementów sąsiadujących

`$("h2 + div")`

wybiera znacznik div umieszczony za
znacznikiem h2

Witryny WWW /67/

Selektory atrybutów



❖ Selektory atrybutów umieszcza się za nazwą elementu

`$("img[alt]")`

wybiera wszystkie znaczniki img
posiadające atrybut alt

❖ Rodzaje selektorów

⌘ `[atrybut]`

⌘ `[atrybut="wartość"]`

⌘ `[atrybut^="wartość"]`

⌘ `[atrybut$="wartość"]`

⌘ `[atrybut*="wartość"]`

Witryny WWW /68/

Filtry /1/



- ❖ :even :odd - pobiera parzyste/nieparzyste elementy
`$("tr:even")`
- ❖ :first :last - pierwszy/ostatni element
`$("p:first")`
- ❖ :not() - elementy nie pasujące do selektora podanego w ()
`$("a:not(.navButton)")`
- ❖ :has() - elementy zawierające inny element
`$("li:has(a)")`
- ❖ :contains() - elementy zawierające podany tekst
`$("a:contains(Kliknij mnie)")`

Witryny WWW /69/

Filtry /2/



- ❖ :hidden - pobiera elementy z właściwością "display: none"
`$("div:hidden").show`
- ❖ :visible - pobiera wszystkie widoczne elementy
`$("div:visible")`

Witryny WWW /70/

Wykorzystanie kolekcji elementów jQuery



- ❖ Przetwarzanie elementów kolekcji w pętli zostało wbudowane w funkcje biblioteki jQuery
`$("#slideshow img").hide();`
pobiera wszystkie obrazki z elementu o identyfikatorze slideshow i dla każdego wywołuje funkcję hide
- ❖ Łącuchy wywołań
**`$("#popup").width(300)
.height(300)
.text("Hello")
.fadeIn(1000);`**

Witryny WWW /71/

Dodawanie treści /1/



```
<div id="container">  
  <div id="errors">  
    <h2>Błędy:</h2>  
  </div>  
</div>
```

- ❖ .html() - zwraca / zastępuje kod html wewnątrz elementu
`alert($("#errors").html());`
`$("#errors").html("<p>Komunikat</p>");`
- ❖ .text() - zwraca/zmienia tekst wewnątrz znacznika (nie akceptuje znaczników html)
`$("#errors h2").text("Brak błędów");`

Witryny WWW /72/

Dodawanie treści /2/



- ❖ `.append()` - dodaje przekazany kod HTML jako ostatni element potomny wcześniej wybranego elementu

`$("#errors").append("<p>W formularzu...</p>");`

```
<div id="container">
  <div id="errors">
    <h2>Błędy:</h2>
    <p>W formularzu znaleziono cztery błędy.</p>
  </div>
</div>
```

- ❖ `.prepend()` - dodaje przekazany kod HTML bezpośrednio za znacznikiem otwierającym wybranego elementu

`$("#errors").prepend("<p>W... </p>");`

- ❖ `.before()` - dodaje przekazany kod HTML poza wybranym elementem przed znacznikiem otwierającym

- ❖ `.after()` - dodaje przekazany kod HTML poza wybranym elementem za znacznikiem zamykającym

Witryny WWW /73/

Zastępowanie / usuwanie elementów



- ❖ `.remove()` - usuwa wybrane elementy
 - ❖ `.replace()` - zastępuje wybrane elementy podanym kodem HTML
- `$("#errors").replace("<p>...</p>");`**
- ❖ `.clone()` - wykonuje kopię wybranego elementu

Witryny WWW /74/

Ustawianie/odczyt klasy i właściwości CSS



- ❖ `.addClass()` - dodaje do elementu podaną nazwę klasy
`$( "a[href^='http://']" ).addClass( "extLink" );`
- ❖ `.removeClass()` - usuwa z elementu podaną nazwę klasy
- ❖ `.toggleClass()` - przełącza klasę elementu
- ❖ `.css()` - ustawia/odczytuje wartość właściwości css wybranego elementu.

Witryny WWW /75/

Ustawianie/odczyt klasy - przykład



```
<head><title>...</title>
<script src="jquery-1.6.3.min.js"></script>
<script>
$(document).ready(function() {
    $("#changestyle").click(function(){
        $("body").toggleClass("altstyle");
    }); // koniec funkcji click
}); // koniec funkcji ready
</script>
<style type="text/css">
    .altstyle {background: yellow}
</style>
</head>
<body>
    <form>
        <input type="button" value="Click me!"
            id="changestyle">
    </form>
</body>
```

Witryny WWW /76/

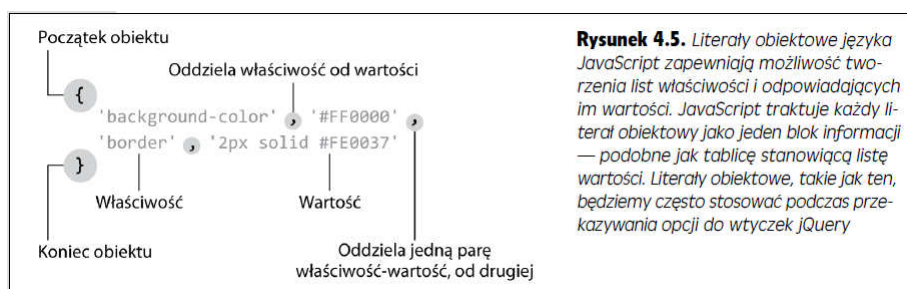
Modyfikacja właściwości CSS - przykład



```
<head><title>...</title>
<script src="jquery-1.6.3.min.js"></script>
<script>
$(document).ready(function() {
  $("#changestyle").click(function(){
    var baseFont=$("#body").css("font-size");
    baseFont=parseInt(baseFont,10);
    $("#body").css("font-size",baseFont*2);
  }); // koniec funkcji click
}); // koniec funkcji ready
</script>
</head>
<body style="font-size: 10pt">
  Przykładowy tekst na stronie.<br>
  <form>
    <input type="button" value="Click me!"
      id="changestyle">
  </form>
</body>
```

Witryny WWW /77/

Literały obiektowe - modyfikacja wielu właściwości



Rysunek 4.5. Literały obiektowe języka JavaScript zapewniają możliwość tworzenia list właściwości i odpowiadających im wartości. JavaScript traktuje każdy literał obiektowy jako jeden blok informacji — podobne jak tablicę stanowiącą listę wartości. Literały obiektowe, takie jak ten, będziemy często stosować podczas przekazywania opcji do wtyczek jQuery

Źródło: David Sawyer McFarland „JavaScript i jQuery – nieoficjalny podręcznik”, Helion 2012

Witryny WWW /78/

Modyfikacja właściwości CSS - przykład



```
<head><title>...</title>
<script src="jquery-1.6.3.min.js"></script>
<script>
$(document).ready(function() {
  $("#changestyle").click(function(){
    $("body").css(
      {"background-color":"yellow",
       "font-size" : "24pt"}); //koniec funkcji css
  }); // koniec funkcji click
}); // koniec funkcji ready
</script>
</head>
<body style="font-size: 10pt">
  Przykładowy tekst na stronie.<br>
  <form>
    <input type="button" value="Click me!"
          id="changestyle">
  </form>
</body>
```

Witryny WWW /79/

Ustawianie/odczyt atrybutów



- ❖ `.attr("nazwa_atrybutu")` - zwraca wartość atrybutu znacznika HTML
`var imageFile=$("#photo1").attr("src");`
- ❖ `.attr("nazwa_atrybutu","wartość")` - ustawia wartość atrybutu znacznika HTML

Witryny WWW /80/

Modyfikacja atrybutów - przykład



```
<head><title>...</title>
<script src="jquery-1.6.3.min.js"></script>
<script>
$(document).ready(function() {
    $("#changephoto").click(function(){
        var file=$("#photo1").attr("src");
        if (file=="weiti1.jpg")
            $("#photo1").attr("src", "awaria.png");
        else
            $("#photo1").attr("src", "weiti1.jpg");
    }); // koniec funkcji click
}); // koniec funkcji ready
</script>
</head>
<body>
    <form>
        <input type="button" value="Zmień" id="changephoto">
    </form>
    
</body>
```

Witryny WWW /81/

Wykonywanie operacji na elementach kolekcji



Początek funkcji each() — Funkcja anonimowa

```
$('.selector').each(function() {
    // your code goes in here
})
```

Początek funkcji anonimowej

Koniec instrukcji

Koniec funkcji each()

Koniec funkcji anonimowej

Rysunek 4.6. Funkcja each() jQuery pozwala przejrzeć całą kolekcję wybranych elementów i na każdym z nich wykonać jakieś operacje. Kluczowym warunkiem, niezbędnym do stosowania tej funkcji jest dobre zrozumienie funkcji anonimowych

Źródło: David Sawyer McFarland „JavaScript i jQuery – nieoficjalny podręcznik”, Helion 2012

Witryny WWW /82/

Modyfikacja atrybutów - przykład



```
<head><title>...</title>
<script src="jquery-1.6.3.min.js"></script>
<script>
$(document).ready(function() {
$("img").each(function(){
    var file=$(this).attr("src");
    $("#wykaz").append(file+"<br>");
}); // koniec funkcji each
}); // koniec funkcji ready
</script>
</head>
<body>
<br>
<br>
<div id="wykaz">
    Wykaz obrazków na stronie:<br>
</div>
</body>
```

Witryny WWW /83/

Programowanie dla sieci WWW



Opracowanie materiałów pomocniczych do kursu:

mgr inż. **Zbigniew Szymański**

e-mail: **Z.Szymanski@ii.pw.edu.pl**



Instytut Informatyki
Politechnika Warszawska



Projekt współfinansowany ze środków Unii Europejskiej w ramach Europejskiego Funduszu Społecznego

Witryny WWW /84/